

Technical Write-Up — Dynamic Portfolio Dashboard

This covers the challenges named in the assignment brief (Section 3) and what was actually built against each one, including the specific findings from testing against the real Yahoo Finance and Google Finance endpoints.

1. API limitations (Section 3, "API Limitations")

CMP — Yahoo Finance (`lib/yahoo.ts`)

Yahoo Finance has no official API. Two unofficial paths exist:

- The `yahoo-finance2` npm package's authenticated `quote()` / `quoteSummary()` calls, which go through a cookie + crumb handshake against `finance.yahoo.com`.
- The plain `/v8/finance/chart/{symbol}` endpoint, which needs no auth at all.

In testing, the authenticated path started returning `429 Too Many Requests` after only a handful of calls — Yahoo appears to rate-limit the crumb-based endpoints much more aggressively than the public chart endpoint. The chart endpoint held up fine under the same load, so `lib/yahoo.ts` is a small `fetch`-based client against it directly rather than a dependency on `yahoo-finance2`. It conveniently also returns a recent daily-close history in the same response, which doubles as sparkline data without a second request.

Symbol resolution. `data/holdings.json` records every BSE holding by its numeric scrip code (e.g. `532174` for ICICI Bank) — how a real brokerage export represents them. Yahoo's `.BO` suffix does not resolve a scrip code directly (`532174.BO` → 404); it needs the actual ticker (`ICICIBANK.BO`). `lib/symbolMap.ts` holds a verified scrip-code → ticker mapping, built by resolving each company name through Yahoo's `/v1/finance/search` endpoint and confirming the result against the chart endpoint — not guessed.

Coverage gaps. Two holdings don't resolve on Yahoo at all in this testing: Savani Financials (zero search results under any name variant tried — likely too thin a micro-cap to be indexed) and LTIMindtree (both `.NS` and `.BO` 404, and it doesn't resolve via search either, despite being a large, liquid stock — reproducible but unexplained). Both fall back to the mock generator for that one field and are tagged `source: 'mock'` rather than left blank or silently guessed.

P/E Ratio & Latest Earnings — Google Finance (`lib/google.ts`)

Google Finance has no API either. Its quote page (`google.com/finance/quote/{symbol}`) statically server-renders a key-stats panel as `<div class="SwQK7">P/E ratio</div><div class="d06ijd">21.32</div>` - style label/value pairs — undocumented, unstable markup with no semantic hooks, exactly the "may break due to site changes" risk the brief names. `lib/google.ts` regex-parses these pairs. If the markup changes and the panel can't be found at all, the fetch fails closed (falls back to mock) rather than returning a wrong number silently.

Two findings from testing that shaped the implementation:

- The stats panel only reliably renders under the `:NSE` suffix for this portfolio's stocks. `ICICIBANK:BOM` returned a full ~1.1MB page with zero instances of the panel's markup; `ICICIBANK:NSE` rendered it every time tested. So every lookup goes through NSE, even for holdings recorded as BSE in our data (all of them are in fact dual-listed, which is what makes the substitution valid rather than just convenient).
- **Latest Earnings has no live path at all.** Google Finance's "Earnings" tab loads its data via a separate client-side request; the initial HTML fetch never contains it — only the tab's label. There's no "try harder" fix for that without reverse-engineering an undocumented internal endpoint, which felt like the wrong side of the line for an unofficial source already being scraped once. So `latestEarnings` is always sourced from the seed data, and that's stated plainly via `earningsSource: 'mock'` in the API response rather than silently implied to be live.

Rate limiting

Both fetchers batch requests at a fixed concurrency (6 for Yahoo, 5 for Google) via `Promise.allSettled`, rather than firing all ~26-29 requests at once. Neither endpoint publishes a rate-limit contract, so this is a courtesy rather than a documented requirement — but `Promise.allSettled` specifically means one bad symbol (delisted, unlisted, a typo) never takes the rest of the batch down with it. On top of that, both `/api/quotes` and `/api/fundamentals` cache their combined response in-memory (`lib/cache.ts`) — 15s for quotes (matching the poll interval, so a page with multiple open tabs doesn't multiply the request count), 4h soft / 24h hard for fundamentals (P/E and earnings don't move intraday).

Data accuracy

Every CMP and P/E value carries a `source: 'live' | 'mock'` tag, surfaced in the UI as a small dot next to the number (hover it for the exact claim — "fetched from the source just now" vs. "the live source was unavailable for this field"). This also distinguishes two different kinds of "no P/E ratio": Google genuinely reporting no number for a stock (`peSource: 'live', peRatio: null` — e.g. Gensol, which showed `-` server-side) from a failed scrape falling back to the seed value (`peSource: 'mock'`). Those are different claims and the UI doesn't conflate them.

2. Asynchronous operations

Both `lib/yahoo.ts` and `lib/google.ts` fetch all symbols in a batch in parallel (`Promise.allSettled`), not sequentially — a sequential loop over ~26 symbols at a few hundred ms each would make every 15s poll take several seconds. `Promise.allSettled` specifically (not `Promise.all`) so a single rejected promise doesn't reject the whole batch.

3. Data transformation

`lib/calculations.ts` is a pure-function pipeline: `enrichHoldings` joins static holdings with live quotes + fundamentals and derives investment/present value/gain-loss/portfolio %; `groupBySector` rolls that

up into sector subtotals; `buildPortfolioSummary` rolls sectors up into the portfolio total. Nothing here cares whether the quotes underneath came from Yahoo or the mock generator — which is also what makes the dashboard's time-travel scrubber work: rewinding just swaps which `Quote[]` array feeds the same pipeline.

4. Performance optimization

- **Caching:** `lib/cache.ts`, a minimal in-memory TTL cache with separate soft/hard expiry (soft = "fetch fresh in the background," not implemented as background refresh here, but the hard expiry means a cache entry is never served indefinitely stale).
- **Memoization:** derived values (`enrichHoldings`, `groupBySector`, `buildPortfolioSummary`, the portfolio health score) are wrapped in `useMemo` keyed on their actual inputs, so a 15s poll tick that doesn't change a given holding's price doesn't force every downstream component to recompute.
- Client polling pauses via the Page Visibility API when the tab isn't focused (`lib/usePortfolioPolling.ts`), so a backgrounded tab doesn't keep hitting the API.

5. Error handling

Three layers, each visible rather than silent:

1. **Per-field** — a failed live fetch for one holding falls back to mock data for just that field, tagged `source: 'mock'`.
2. **Per-source** — if *any* holding's Yahoo or Google fetch failed this tick, the header's status badge switches from "Live" to "Degraded" and names which source.
3. **Whole-request** — if `/api/quotes` itself fails (network down, server error), the UI shows "Can't reach the server — showing last known data" rather than clearing the table.

6. Security

No API keys are involved — both sources are unauthenticated public endpoints — so there's nothing to leak client-side. Both fetchers run exclusively in Next.js Route Handlers (`app/api/*/route.ts`), never in client components, so even if that changed in the future the fetch calls (and any future credentials) would stay server-side by construction.

7. What's still mock-only, and why

- "Latest Earnings" — no live path exists via either source for the reasons above; it's always the seeded value, honestly tagged.
- The suspicious-duplicate P/E quirk (four holdings sharing an identical mock P/E) is a deliberately seeded demonstration of a scraping artifact. It's switched off once a real, distinct P/E comes back from Google — real data for four different companies isn't going to coincidentally collide.

- `DATA_MODE=mock` remains available as an escape hatch for an offline or fast demo, since both live sources depend on outbound network access that may not be available in every environment.